تقرير العمل التطبيقي الأول — UniAccounts

نماذج التصميم (Design Patterns) بلغة Java — النواة وأنماط الإنشاء

جامعة الشهيد حمه لخضر – الوادي | 2025/2026

أعضاء الفوج: .....

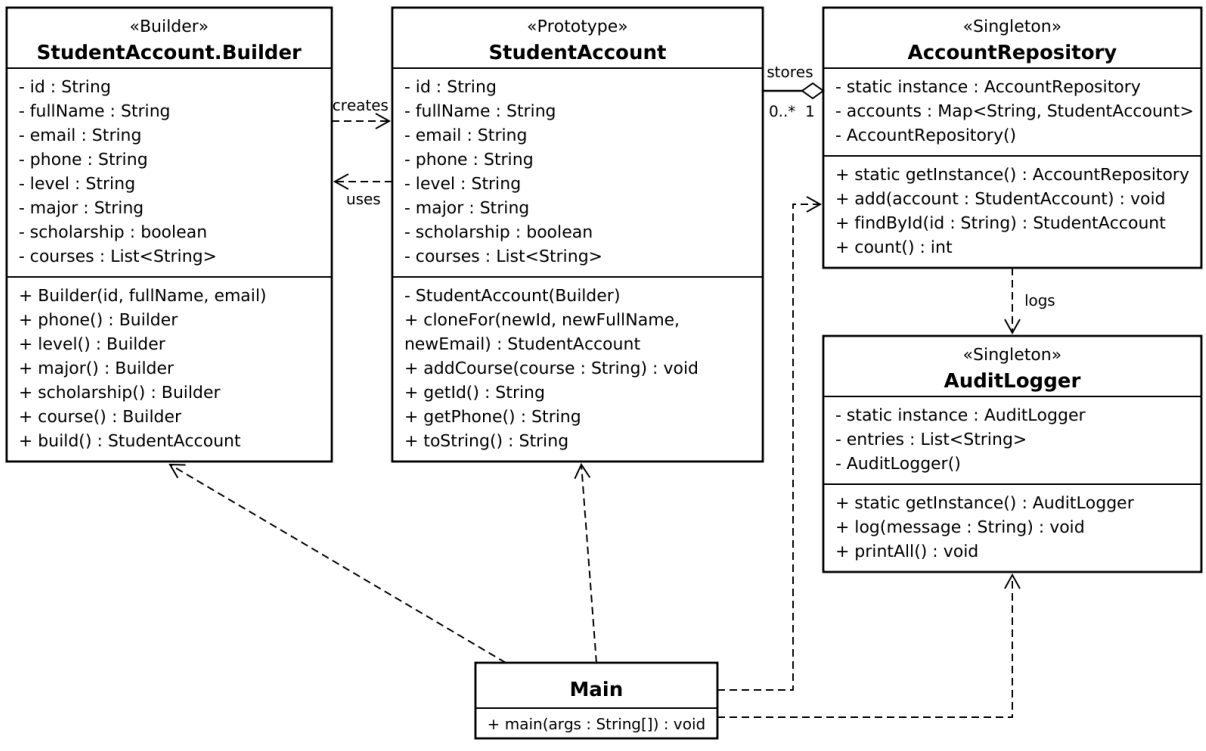
المستودع: github.com/tahartamma-cyber/UniAccounts | الوسم (tag): work1

1 - فكرة العمل

نواة نظام UniAccounts لإدارة حسابات الطلبة: ننشئ حساب الطالب بمعلوماته، ونحفظه في مستودع واحد، ونسجّل العمليات في سجل تدقيق واحد، ونُنشئ حسابات الدفعة الجديدة بنسخ قالب جاهز.

النمط	الصنف	المشكلة التي يحلها
Builder	StudentAccount.Builder	حساب الطالب له 8 حقول، بعضها إجباري وبعضها اختياري، فبناؤه بمنشئ واحد غير مقروء.
Singleton	AccountRepository AuditLogger	يجب أن يوجد مستودع واحد وسجل تدقيق واحد في البرنامج كله.
Prototype	StudentAccount.cloneFor	طلبة الدفعة يلتحقون بنفس التخصص والمستوى والمقررات، فنستنسخ قالباً ونغيّر الهوية فقط.

2 - مخطط الأصناف (UML)



الشكل 1: مخطط الأصناف للعمل الأول (الملف docs/uml.png)

مفتاح القراءة: الأسهم المتقطعة تعني «ينشئ» أو «يستعمل» أو «يسجّل في»، والسهم ذو الماسة الفارغة يعني أن المستودع يخزّن حسابات (من صفر إلى عدد غير محدود).

3 - الأنماط المستعملة

3.1 نمط Builder (الباني)

أين استعملته؟

في الصنف المتداخل StudentAccount.Builder. الحقول الإجبارية الثلاثة (id، fullName، email) تُمرَّر إلى منشئ الـ Builder، والحقول الاختيارية (phone، level، major، scholarship، course) تُضبط بدوال متسلسلة، ثم تُنشئ build() الحساب بعد التحقق من القواعد: id وfullName غير فارغين، وemail يحتوي @، وlevel واحد من L1 وL2 وL3 و M1 وM2. منشئ StudentAccount خاص (private)، فلا يُنشأ الحساب إلا عبر الـ Builder.

```
StudentAccount template = new StudentAccount.Builder(
        "TEMPLATE", "Template", "template@univ.dz")
    .level("L3").major("Informatique")
    .course("POO").course("BD").course("Reseaux")
    .build();
```

ما المشكلة التي حلها؟

الحساب له ثمانية حقول، ومنشئ بثمانية وسائط يجعل الكود غير مقروء ويسهل فيه خلط ترتيب القيم، كما أن الحقول الاختيارية كانت ستُمرَّر null. مع Builder نكتب فقط ما نحتاجه وبأسماء واضحة.

ماذا لو لم أستعمله؟

سنكتب مثلاً:

```
new StudentAccount("101", "Amine", "amine@univ.dz",
                   null, "L3", "Informatique", false, courses);
```

وهنا قد تُبدَّل قيمتا level وmajor دون أن يظهر أي خطأ، كما يتشتت التحقق من صحة البيانات في عدة أماكن بدل أن يكون في build() وحدها، فقد تُنشأ حسابات ببريد خاطئ أو مستوى غير موجود.

3.2 نمط Singleton (المفرد)

أين استعملته؟

في صنفين: AccountRepository (مستودع الحسابات) وAuditLogger (سجل التدقيق). لكل منهما حقل ثابت instance، ومنشئ خاص، ودالة getInstance() تنشئ الكائن عند أول طلب فقط، وهذا ما يسمى Lazy Initialization، ثم تعيد نفسه دائماً. والصنفان يتعاونان: دالة add() في المستودع ترفض الـ id المكرر وتكتب في السجل.

```
public static AuditLogger getInstance() {
    if (instance == null) instance = new AuditLogger();
    return instance;
}
```

ما المشكلة التي حلها؟

المطلوب مستودع واحد يحفظ كل الحسابات وسجل واحد يسجّل كل العمليات، ليكون الـ id فريداً في النظام كله ويكون السجل كاملاً.

ماذا لو لم أستعمله؟

لو أنشأ كل جزء من البرنامج مستودعه الخاص بـ new لتوزعت الحسابات على عدة مستودعات، ولاستطاع حسابان في مستودعين مختلفين أن يحملا الـ id نفسه، ولتفرّق سجل التدقيق على عدة نسخ.

> **حدّ معروف:** الدالة getInstance() في صورتها الحالية غير آمنة في بيئة الخيوط (Threads)، وقد تُنشأ نسختان لو استُدعيت من خيطين في اللحظة نفسها. مشروعنا يعمل بخيط واحد فلا تظهر المشكلة.

3.3 نمط Prototype (النموذج الأولي)

أين استعملته؟

في الدالة `StudentAccount.cloneFor(newId, newFullName, newEmail)`. تبني الدالة حساباً جديداً مستقلاً يأخذ المستوى والتخصص والمنحة من القالب، وتنسخ المقررات نسخاً عميقاً (قائمة جديدة)، فيمكن إضافة مقرر جديد لكل نسخة دون التأثير على غيرها. في `Main` أنشأنا قالباً لدفعة `L3` معلوماتية، ولم نضفه إلى المستودع، ثم استنسخنا منه ثلاثة طلبة.

```
StudentAccount a = template.cloneFor("101", "Amine Ben", "amine@univ.dz");
StudentAccount b = template.cloneFor("102", "Sara Kaci", "sara@univ.dz");
b.addCourse("IA");   // only Sara gets IA
```

ما المشكلة التي حلها؟

طلبة الدفعة يلتحقون بنفس التخصص والمستوى والمقررات، وبناء كل حساب من الصفر تكرار لنفس الإعدادات. بالنسخ نغيّر الهوية فقط (الرقم والاسم والبريد).

ماذا لو لم أستعمله؟

سنكرر سلسلة الـ `Builder` الكاملة لكل طالب. ولو نسخنا القائمة نسخاً سطحياً (إعادة استعمال المرجع نفسه) لتشارك الطلبة قائمة المقررات، فإضافة `IA` لسارة ستظهر عند الجميع وعند القالب أيضاً. كما أن استدعاء `build()` من جديد لكل نسخة يكرر قواعد التحقق.

4 - نتيجة التشغيل

شغّلنا الأمر `java -cp out uniaccounts.Main` فكانت النتيجة التالية:

```
PS C:\Users\HSN\OneDrive\Desktop\UniAccounts> javac -d out -sourcepath src src/uniaccounts/Ma
in.java src/uniaccounts/ValidationDemo.java
PS C:\Users\HSN\OneDrive\Desktop\UniAccounts> java -cp out uniaccounts.Main
101 | Amine Ben | amine@univ.dz | L3 | Informatique | scholarship=false | [POO, BD, Reseaux]
102 | Sara Kaci | sara@univ.dz | L3 | Informatique | scholarship=false | [POO, BD, Reseaux, I
A]
103 | Yacine Omar | yacine@univ.dz | L3 | Informatique | scholarship=false | [POO, BD, Reseau
x]
TEMPLATE | Template | template@univ.dz | L3 | Informatique | scholarship=false | [POO, BD, Re
seaux]
Count = 3
--- Audit log ---
1. CREATED account 101
2. CREATED account 102
3. CREATED account 103
PS C:\Users\HSN\OneDrive\Desktop\UniAccounts>
```

الشكل 2: نتيجة تشغيل `Main` (الملف `docs/capture_main.png`)

ماذا تثبت هذه النتيجة؟

المشاهدة	الدليل على
الحسابات 101 و102 و103 تحمل `L3` و`Informatique` والمقررات `POO, BD, Reseaux`	**Prototype:** نُسخت من القالب، وتغيّرت الهوية فقط.
المقرر `IA` ظهر عند الحساب 102 فقط، ولم يظهر عند القالب ولا عند غيره	**النسخ العميق:** لكل حساب قائمة مقررات مستقلة.
`Count = 3`	القالب لم يُضَف إلى المستودع، وإلا لكان العدد 4.

المشاهدة	الدليل على
سجل التدقيق فيه ثلاث عمليات CREATED	**Singleton:** المستودع يكتب في السجل الوحيد نفسه الذي طبعه Main.
إنشاء القالب والحسابات تم بلا أخطاء	**Builder:** التحقق في build() قبل بناء الحساب.

5 - اختبار رفض البيانات الخاطئة

أضفنا الصنف ValidationDemo لإثبات أن البرنامج يرفض البيانات الخاطئة ولا يقبلها بصمت. شغّلناه بالأمر java -cp out uniaccounts.ValidationDemo فكانت النتيجة التالية:

```
PS C:\Users\HSN\OneDrive\Desktop\UniAccounts> java -cp out uniaccounts.ValidationDemo
Builder  -> email invalide
Builder  -> level invalide
Singleton -> id deja existant: 101
--- Audit log ---
1. CREATED account 101
2. REJECTED duplicate id 101
PS C:\Users\HSN\OneDrive\Desktop\UniAccounts>
```

الشكل 3: نتيجة رفض البيانات الخاطئة (الملف docs/capture_validation.png)

السطر الظاهر في النتيجة	ما يثبته
Builder -> email invalide	بريد بلا @: رفضته build() ولم يُنشأ الحساب.
Builder -> level invalide	المستوى L9 ليس من L1 وL2 وL3 وM1 وM2، فرُفض.
Singleton -> id deja existant: 101	المستودع الوحيد رفض حساباً ثانياً بنفس id.
REJECTED duplicate id 101	السجل الوحيد سجّل الرفض أيضاً، أي أن الصنفين يتعاونان.

ملاحظة: يبدأ سجل التدقيق من الرقم 1 من جديد لأن ValidationDemo برنامج مستقل عن Main، وكل برنامج له مستودعه وسجله الوحيدان.

6 - تعاون الأنماط الثلاثة

يبني Main القالب بـ **Builder**، ثم يستنسخ منه ثلاثة حسابات بـ **Prototype**، ثم يضيفها إلى المستودع **Singleton** الذي يسجّل كل عملية في سجل التدقيق **Singleton**.

7 - تنظيم المشروع وطريقة التشغيل

الأصناف موزعة على الحزم (packages) التالية:

```
src/uniaccounts/
  Main.java, ValidationDemo.java
  model/StudentAccount.java        (Builder + Prototype)
  repository/AccountRepository.java (Singleton)
  util/AuditLogger.java            (Singleton)
docs/                              (UML + screenshots + how to run)
```

التشغيل من جذر المشروع، والمتطلب هو JDK 17 أو أحدث، بالأوامر التالية:

```
javac -d out -sourcepath src src/uniaccounts/Main.java
src/uniaccounts/ValidationDemo.java
java -cp out uniaccounts.Main
java -cp out uniaccounts.ValidationDemo
```

8 - المصادر

المصدر 1: موقع Refactoring.Guru — قسم أنماط التصميم، وقد رجعنا فيه إلى صفحات الأنماط الثلاثة.

```
https://refactoring.guru/design-patterns
```

المصدر 2: التوثيق الرسمي للغة Java من شركة Oracle.

```
https://docs.oracle.com/javase/
```

المصدر 3: دروس مقياس نماذج التصميم (Design Patterns)، جامعة الشهيد حمه لخضر – الوادي، 2025/2026.